TÁCH TỪ
Word segmentation

Tách từ

- Mục đích: xác định ranh giới của các từ trong câu.
- Là bước xử lý quan trọng đối với các hệ thống XLNNTN, đặc biệt là đối với các ngôn ngữ đơn lập, ví dụ: âm tiết Trung Quốc, âm tiết Nhật, âm tiết Thái, và tiếng Việt.
- Với các ngôn ngữ đơn lập, một từ có thể có một hoặc nhiều âm tiết.

→ Vấn đề của bài toán tách từ là khử được sự nhập nhằng trong ranh giới từ, xuất hiện các từ mới không có trong từ điển

→ http://www.loria.fr/~lehong/tools/vnTokenizer.php

Từ vựng tiếng Việt

- Tiếng Việt là ngôn ngữ không biến hình
- Âm tiết (còn gọi là tiếng, chữ) là đơn vị phát âm nhỏ nhất nhưng hoàn chỉnh trong ngôn ngữ, đóng vai trò nền tảng trong việc hình thành từ và câu
- Từ điển từ tiếng Việt (Vietlex): >40.000 từ, trong đó:
  - 15.69% các từ trong từ điển là từ đơn
  - 70.72% từ ghép có 2 âm tiết
  - 13.59% từ ghép ≥ 3 âm tiết
  - 1.04% từ ghép ≥ 4 âm tiết

Độ dài	Số từ	Tỉ lệ
1	6,303	15.69
2	28,416	70.72
3	2,259	5.62
4	2,784	6.93
5	419	1.04
Tổng	40,181	100

Qui tắc cấu tạo từ tiếng Việt

- Từ đơn: *dùng một âm tiết* làm một từ.
  - Ví dụ: *tôi, bác, người, cây, hoa, đi, chạy, vì, đã, à, nhỉ, nhé*, ...
- Từ ghép: tổ hợp (ghép) các âm tiết lại, giữa các âm tiết đó có quan hệ về nghĩa với nhau.
  - *Từ ghép đẳng lập*: các thành tố cấu tạo có quan hệ bình đẳng với nhau về nghĩa.
    - Ví dụ: *chợ búa*, *bếp núc*
  - *Từ ghép chính phụ*: các thành tố cấu tạo này phụ thuộc vào thành tố cấu tạo kia. Thành tố phụ có vai trò phân loại, chuyên biệt hoá và sắc thái hoá cho thành tố chính.
    - Ví dụ: *tàu hoả, đường sắt, xấu bụng, tốt mã, ngay đơ, thẳng tắp, sưng vù*, ...

Qui tắc cấu tạo từ tiếng Việt

- Từ láy: các yếu tố cấu tạo có thành phần ngữ âm được lặp lại; nhưng vừa lặp vừa biến đổi. Một từ được lặp lại cũng cho ta từ láy.
- Biến thể của từ: được coi là dạng lâm thời biến động hoặc dạng "lời nói" của từ.
  - Rút gọn một từ dài thành từ ngắn hơn
    - ki-lô-gam → ki lô/ kí lô
  - Lâm thời phá vỡ cấu trúc của từ, phân bố lại yếu tố tạo từ với những yếu tố khác ngoài từ chen vào. Ví dụ:
    - khổ sở → lo khổ lo sở
    - ngặt nghẽo → cười ngặt cười nghẽo
    - danh lợi + ham chuộng → ham danh chuộng lợi

Qui tắc cấu tạo từ tiếng Việt

- Các diễn tả gồm nhiều từ (vd, “bởi vì”) cũng được coi là 1 từ
- Tên riêng: tên người và vị trí được coi là 1 đơn vị từ vựng
- Các mẫu thường xuyên: số, thời gian

Các hướng tiếp cận

- Tiếp cận dựa trên từ điển
- Tiếp cận theo phương pháp thống kê
- Kết hợp hai phương pháp trên.

Các phương pháp

- So khớp từ dài nhất (Longest Matching)
- Học dựa trên sự cải biến (Transformation-based Learning – TBL)
- Chuyển đổi trạng thái trọng số hữu hạn (Weighted Finite State Transducer – WFST)
- Entropy cực đại (Maximum Entropy – ME)
- Học máy sử dụng mô hình Markov ẩn (Hidden Markov Models- HMM)
- Học máy sử dụng vectơ hỗ trợ (Support Vector Machines)
- Trường điều kiện ngẫu nhiên (Conditional Random Field)
- Kết hợp một số phương pháp trên

Tiếp cận dựa trên từ điển

- **Xây dựng từ điển**
  - Mỗi mục từ lưu thông tin:
    - từ,
    - từ loại,
    - nghĩa loại (ví dụ: danh từ chỉ người)
  - Tổ chức sao cho tốn ít bộ nhớ và thuận tiện trong việc tìm kiếm
- **Mã hóa từ điển**: Từ loại và nghĩa loại kiểu byte được lưu dưới dạng một ký tự.
  - VD: danh từ: 112 – p

Tiếp cận dựa trên từ điển

- Thuật toán so khớp từ dài nhất
- Yêu cầu:
  - Từ điển
  - Chuỗi đầu vào đã tách các dấu câu và âm tiết.
- Tư tưởng: thuật toán tham lam
  - Đi từ trái sang phải hoặc từ phải sang trái, lấy các từ dài nhất có thể, dừng lại khi duyệt hết
  - Độ phức tạp tính toán: O(n.V)
    - n: Số âm tiết trong chuỗi
    - V: Số từ trong từ điển

Tiếp cận dựa trên từ điển

- Thuật toán so khớp từ dài nhất
  - **Bắt đầu**
  - (1) Chuỗi đầu vào [$w_0 w_1 \ldots w_{n-1}$]
  - (2) *words* ← []
  - (3) $s \leftarrow 0$
  - (4) $e \leftarrow n$
  - (5) Khi [$w_s \ldots w_e$] chưa là một từ: $e \leftarrow e - 1$
  - (6) *words* ← *words* + [$w_s \ldots w_e$]
  - (7) $s \leftarrow e + 1$
  - (8) Nếu $e < n$: quay lại bước (4)
  - (9) Lấy ra chuỗi đã tách từ *words*
  - **Kết thúc**

Tiếp cận dựa trên từ điển

- Thuật toán so khớp từ dài nhất
  - **Ưu điểm**
    - Cài đặt đơn giản
    - Độ phức tạp tính toán hợp lý
    - Không yêu cầu dữ liệu huấn luyện
  - **Nhược điểm:**
    - Phụ thuộc vào từ điển
    - Chưa giải quyết được vấn đề nhập nhằng

Tiếp cận dựa trên từ điển

- Bài tập
  - Cài đặt thuật toán so khớp từ dài nhất bằng Python
  - Một số mẫu thử:
    - Thời khóa biểu đang được cập nhật
    - Môn học xử lý ngôn ngữ tự nhiên
    - Ông già đi nhanh quá
    - Con ngựa đá con ngựa đá
    - Học sinh học sinh học

Tiếp cận dựa trên từ điển

```
1  def tokenizer(text, dict, is_show=False):
2      print ("input:", text)
3      print ()
4      input = text.split(" ") #[w_0, w_1,..., w_n-1]
5      words = []
6      s = 0
7      while True:
8          e = len(input)
9          while e > s:
10             tmp_word = input[s:e] # [w_s ... w_e]
11             is_word = ""
12             for item in tmp_word:
13                 is_word += item + " "
14             is_word = is_word[:-1] #Loại bỏ dấu cách thừa ở cuối
15             e -= 1
16             # print (is_word)
17             if is_word.lower() in dict:
18                 words.append(is_word) # words <- words + [w_s ... w_e]
19                 break
```

Tiếp cận dựa trên từ điển

```
19                  break
20              if e == s:
21                  words.append(is_word) # words <- words + first_word
22                  break
23          if e >= len(input):
24              break
25          #Hiển thị quá trình tách từ
26          if is_show:
27              print("s =", s)
28              print("e =", e)
29              print(words[len(words) - 1])
30              print("-" * 100)
31          s = e + 1
32      output = ""
33      for item in words:
34          output += item.replace(" ", "_")
35          output += " "
36      output = output[:-1]
37      return output
```

Tiếp cận dựa trên từ điển

```
39  if __name__ == "__main__":
40      ex1 = "thời khóa biểu đang được cập nhật"
41      ex2 = "môn học xử lý ngôn ngữ tự nhiên"
42      ex3 = "con ngựa đá con ngựa đá"
43      ex4 = "học sinh học sinh học"
44
45      #Từ điển
46      dict = {"thời khóa biểu": 0,"đang": 1,"được": 2,"cập nhật": 3,
47              "môn học": 4,"môn": 5; "học": 6,"xử lý": 7,"ngôn ngữ": 8,
48              "tự nhiên": 9,"con": 10, "con ngựa": 11, "ngựa": 12,
49              "đá": 13, "học": 13, "học sinh": 14, "sinh học": 15,
50              "dân tộc": 16, "viện trưởng": 17, "giáo viên": 18,
51              "đạo diễn": 19,"xứ sở": 20, "nguồn lực": 21, "thủ đô": 22,
52              "số lượng": 23, "thuần nhất": 24, "môi giới": 25,
53              "đơn giản": 26, "tiến bộ": 27, "chính sách": 28,
54              "thường xuyên": 29, "tình yêu": 30; }
55
56      test1 = tokenizer(ex2, dict)
57
58      print ("output:", test1)
```

Tiếp cận dựa trên từ điển

- **Cách tách từ đơn giản**
  - Phát hiện các mẫu thông thường như tên riêng, chữ viết tắt, số, ngày tháng, địa chỉ email, URL, … sử dụng biểu thức chính qui
  - Chọn chuỗi âm tiết dài nhất từ vị trí hiện tại và có trong từ điển, chọn cách tách có ít từ nhất

  → Hạn chế: có thể đưa ra cách phân tích không đúng.

  → Giải quyết: liệt kê tất, có một chiến lược để chọn cách tách tốt nhất. Nếu phân tích cú pháp cho ra cây đúng thì đó là cách tách đúng.

Tách từ sử dụng Biểu thức chính qui

- Là một khuôn mẫu được so sánh với một chuỗi
- Các ký tự đặc biệt:
- * - bất cứ chuỗi ký tự nào, kể cả chuỗi rỗng
- x - ít nhất 1 ký tự
- + - chuỗi trong ngoặc xuất hiện ít nhất 1 lần
- Ví dụ:
  - Email: x@x(.x)+
  - dir *.txt
  - '*John' -> 'John', 'Ajohn', "Decker John"
- Biểu thức chính quy được sử dụng đặc biệt nhiều trong:
  - Phân tích cú pháp
  - Xác nhận tính hợp lệ của dữ liệu
  - Xử lý chuỗi
  - Tách dữ liệu và tạo báo cáo

Automat hữu hạn

- Lớp ngôn ngữ chính qui, được đoán nhận bởi máy ảo, gọi tên là automat hữu hạn.
  - Automat hữu hạn đơn định (Deterministic Finite Automat a– DFA
  - Automat hữu hạn không đơn định (Nondeterministic Finite Automat a–NFA)
  - Automat hữu hạn không đơn định, chấp nhận phép truyền rỗng (ε-NFA)
- Một bài toán trong automat là nhận diện chuỗi *w* có thuộc về ngôn ngữ *L* hay không.
- Chuỗi nhập được xử lý tuần tự từng ký hiệu một từ trái sang phải.
- Trong quá trình thực thi, automat cần phải nhớ thông tin đã qua xử lý.

Automat hữu hạn – Định nghĩa

Một *otomat hữu hạn đơn định* (*Deterministic Finite Automata-DFA*) là một bộ năm:

$$A = < Q, \Sigma, \delta, q_0, F >$$

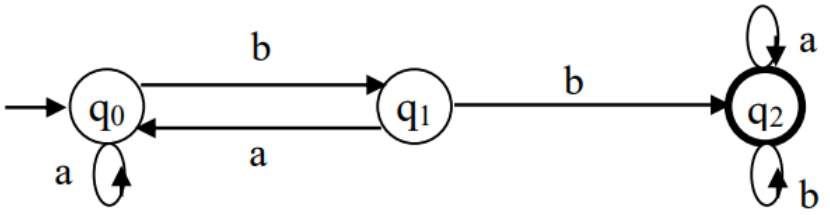
trong đó:

- Q là một tập hữu hạn khác rỗng, được gọi là *tập các trạng thái (set of states)*;
- Σ là một bảng chữ cái, được gọi là bảng chữ vào *(input alphabet);*
- δ: $D \rightarrow Q$, là một ánh xạ từ $D \subseteq Q \times \Sigma$ vào Q, được gọi là *hàm chuyển trạng thái (transition function)*;
- $q_0 \in Q$, được gọi là *trạng thái khởi đầu (starting state);*
- $F \subseteq Q$ được gọi là tập *trạng thái kết thúc (final states).*

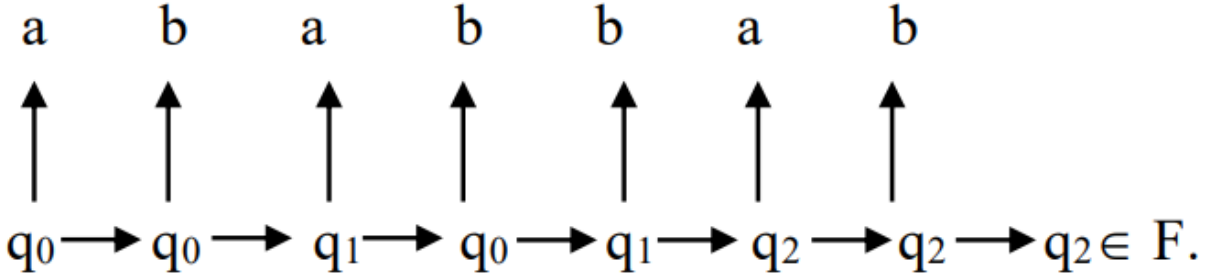
Ví dụ

- Cho Automat A_1
  - $A_1 = <\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\}>$,

Trạng thái	Ký hiệu vào a	b
q_0	q_0	q_1
q_1	q_0	q_2
q_2	q_2	q_2



Dãy trạng thái của otomat A_1 trong quá trình đoán nhận xâu vàoα = ababbab là:



Ngôn ngữ được đoán nhận bởi Automat hữu hạn

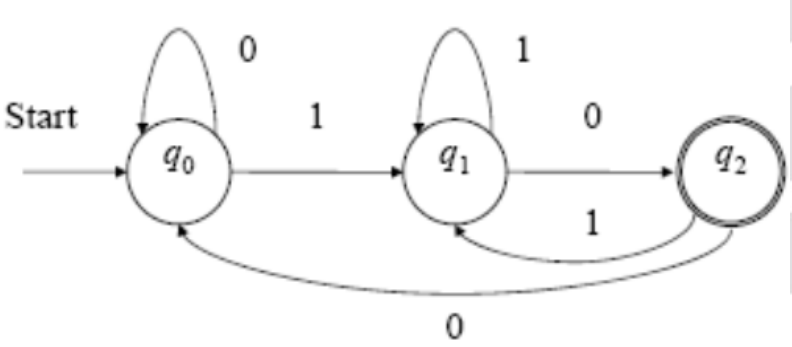
❑ Định nghĩa:

Cho otomat hữu hạn đơn định $A = <Q, \Sigma, \delta, q_0, F>$, *và một xâu* $\omega \in \Sigma^*$. *Ta nói* ω *được đoán nhận bởi A nếu* $\delta(q_0, \omega) \in F$. *Tập hợp tất cả các từ được đoán nhận bởi otomat A gọi là ngôn ngữ được đoán nhận bởi otomat A, và ký hiệu là T(A), vậy:*

$$T(A) = \{\omega \in \Sigma^* \mid \delta(q_0, \omega) \in F\}$$

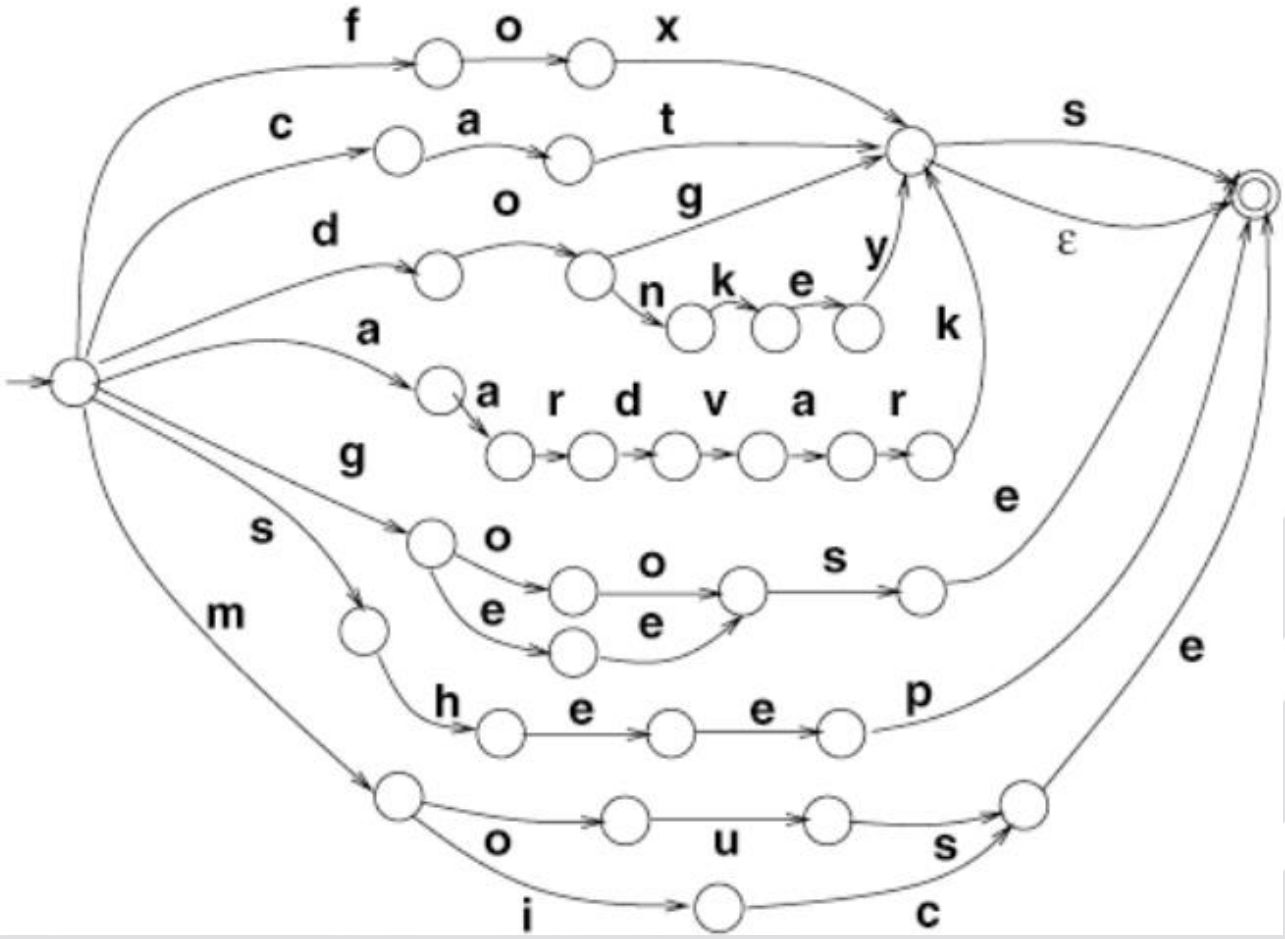
❑ Ví dụ về Automat hữu hạn

- $L = \{w \in \{0, 1\}^* \mid w$ kết thúc bằng chuỗi con 10$\}$.



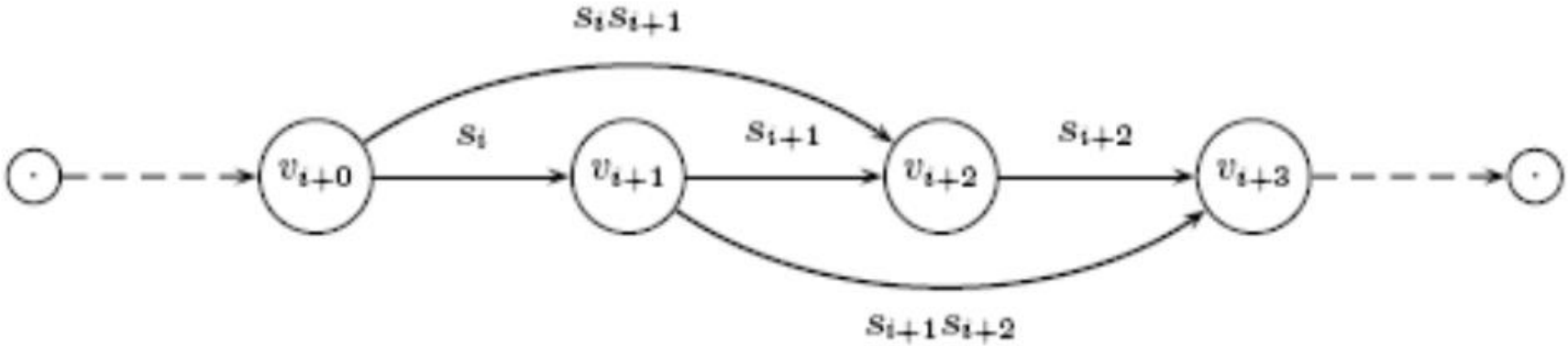
Automat hữu hạn

- Automat hữu hạn cho các từ tiếng Anh



Lựa chọn cách tách từ

- Biểu diễn đoạn bằng chuỗi các âm tiết $s_1\ s_2\ \ldots\ s_n$
- Trường hợp nhập nhằng thường xuyên nhất là 3 từ liền nhau $s_1s_2s_3$ trong đó s_1s_2 và s_2s_3 đều là từ.
  - Ví dụ: Học sinh học sinh học, Mùi thơm ngát gian phòng



- Biểu diễn 1 đoạn bằng đồ thị có hướng tuyến tính $G = (V, E)$, $V = \{v_0, v_1, \ldots, v_n, v_{n+1}\}$
- Nếu các âm tiết $s_{i+1}, s_{i+2}, \ldots, s_j$ tạo thành 1 từ -> trong G có cạnh (v_i, v_j)
- Các cách tách từ = các đường đi ngắn nhất từ v_0 đến v_{n+1}

Thuật toán

- Thuật toán xây dựng đồ thị cho chuỗi $s_1 s_2 \ldots s_n$

1: $V \leftarrow \varnothing$;
2: **for** $i = 0$ **to** $n + 1$ **do**
3: $\quad V \leftarrow V \cup \{v_i\}$;
4: **end for**
5: **for** $i = 0$ **to** n **do**
6: $\quad$ **for** $j = i$ **to** n **do**
7: $\quad\quad$ **if** (accept(A_W, $s_i \cdots s_j$)) **then**
8: $\quad\quad\quad E \leftarrow E \cup \{(v_i, v_{j+1})\}$;
9: $\quad\quad$ **end if**
10: $\quad$ **end for**
11: **end for**
12: **return** $G = (V,E)$;

- accept(A, s): automat A nhận xâu vào s

Phân giải nhập nhằng

- Mô hình ước lượng khả năng tối đa (Maximum Likelihood Estimation)
- Xác suất xâu s?

Xác suất chuỗi s

- Xác suất xuất hiện của từ *w* sau một chuỗi *h* các từ đã biết trước đó ký hiệu là $P(w|h)$
- Ví dụ ta đã biết chuỗi *h* là: "Its water is so transparent that". Ta cần tính xác suất từ tiếp theo là từ "the" ký hiệu là: $P(the|Its\ water\ is\ so\ transparent\ that)$.
- Xác suất này được tính toán theo công thức

$$P(the|Its\ water\ is\ so\ transparent\ that) = \frac{C(Its\ water\ is\ so\ transparent\ that\ the)}{C(Its\ water\ is\ so\ transparent\ that)}$$

Xác suất chuỗi s

$$P(the|Its\ water\ is\ so\ transparent\ that) = \frac{C(Its\ water\ is\ so\ transparent\ that\ the)}{C(Its\ water\ is\ so\ transparent\ that)}$$

- Với tập ngữ liệu đủ lớn ví dụ như web, sách, báo, trong nhiều trường hợp thì công thực trên có thể tính toán được tuy nhiên chi phí tính toán lớn
- Do ngôn ngữ có tính tiến hóa, một số trường hợp đặc biệt ví dụ như những từ mới xuất hiện, ta không thể tính toán chính xác được xác suất nêu trên
- Do đó phải có một phương pháp tổng quát

Xác suất chuỗi s

❑ Cho xâu s = (w_1 , w_2 , …, w_m)

- Xác suất xâu s:

$$P(s) = \prod_{i=1}^{m} P(w_i|w_1^{i-1}) \approx \prod_{i=1}^{m} P(w_i|w_{i-n+1}^{i-1})$$

- $P(w_i|w_1^{i-1})$: xác suất w_i khi có i-1 từ trước đó
- n = 2: bigram; n = 3: trigram

Phân giải nhập nhằng

- Khi n = 2, tính giá trị $P(w_i|w_{i-1})$ lớn nhất maximum likelihood (ML)

$$P_{ML}(w_i|w_{i-1}) = \frac{P(w_{i-1}w_i)}{P(w_{i-1})} = \frac{c(w_{i-1}w_i)/N}{c(w_{i-1})/N} = \frac{c(w_{i-1}w_i)}{c(w_{i-1})}$$

- $c(s)$: số lần xâu s xuất hiện; N: tổng số từ trong tập huấn luyện
- Khi dữ liệu huấn luyện nhỏ hơn kích cỡ toàn bộ tập dữ liệu → P ~ 0

=> Sử dụng kỹ thuật làm trơn

Kỹ thuật làm trơn

$$\widehat{P}(w_i|w_{i-1}) = \lambda_1 P_{ML}(w_i|w_{i-1}) + \lambda_2 P_{ML}(w_i)$$

với $\lambda_1 + \lambda_2 = 1$ và $\lambda_1, \lambda_2 \geq 0$

$P_{ML}(w_i) = c(w_i)/N$

Với tập thử nghiệm $T = \{s_1, s_2, \ldots, s_n\}$, xác suất P(T) của tập thử:

$$P(T) = \prod_{i=1}^{n} P(s_i)$$

Xác định các giá trị lamda

Từ tập dữ liệu mẫu, định nghĩa $C(w_{i-1},w_i)$ là số lần (w_{i-1}, w_i) xuất hiện trong tập mẫu. Ta cần chọn λ_1 λ_2 để làm cực đại giá trị

$$L(\lambda_1, \lambda_2) = \sum_{w_{i-1}, w_i} C(w_{i-1}, w_i) \log_2 \hat{P}(w_i | w_{i-1})$$

với $\lambda_1 + \lambda_2 = 1$ và $\lambda_1, \lambda_2 \geq 0$

Thuật toán

Thuật toán 2. Xác định giá trị λ

1: $\lambda_1 \leftarrow 0.5, \lambda_2 \leftarrow 0.5;$

2: $\epsilon \leftarrow 0.01;$

3: **repeat**

4: $\widehat{\lambda}_1 \leftarrow \lambda_1, \quad \widehat{\lambda}_2 \leftarrow \lambda_2;$

5: $c_1 \leftarrow \sum_{w_{i-1}, w_i} \frac{C(w_{i-1}, w_i)\lambda_1 P_{ML}(w_i|w_{i-1})}{\lambda_1 P_{ML}(w_i|w_{i-1}) + \lambda_2 P_{ML}(w_i)};$

6: $c_2 \leftarrow \sum_{w_{i-1}, w_i} \frac{C(w_{i-1}, w_i)\lambda_2 P_{ML}(w_i)}{\lambda_1 P_{ML}(w_i|w_{i-1}) + \lambda_2 P_{ML}(w_i)};$

7: $\lambda_1 \leftarrow \frac{c_1}{c_1 + c_2}, \quad \lambda_2 \leftarrow 1 - \widehat{\lambda}_1;$

8: $\widehat{\epsilon} \leftarrow \sqrt{(\widehat{\lambda}_1 - \lambda_1)^2 + (\widehat{\lambda}_2 - \lambda_2)^2};$

9: **until** $(\widehat{\epsilon} \leq \epsilon);$

10: **return** $\lambda_1, \lambda_2;$

Phân giải nhập nhằng

- Cách tiếp cận lai
  - <Phuong Le-Hong et al., A hybrid approach to word segmentation of Vietnamese texts, Proceedings of the 2nd International Conference on Language and Automat Theory and Applications, LATA 2008, Tarragona, Spain, 2008.>
  - Kết hợp phân tích automat hữu hạn + biểu thức chính quy + so khớp từ dài nhất + thống kê (để giải quyết nhập nhằng)

Kết quả

- Sử dụng tập dữ liệu gồm 1264 bài trong báo Tuổi trẻ, có 507,358 từ
- Lấy $\varepsilon = 0.03$, các giá trị λ hội tụ sau 4 vòng lặp

Step	λ_1	λ_2	ϵ
0	0.500	0.500	1.000
1	0.853	0.147	0.499
2	0.952	0.048	0.139
3	0.981	0.019	0.041
4	0.991	0.009	0.015

- Độ chính xác = số từ hệ thống xác định đúng/tổng số từ hệ thống xác định = 95%

Một số công cụ tách từ

- JvnSegmenter (Nguyễn Cẩm Tú) : CRF
  http://jvnsegmenter.sourceforge.net
- VnTokenizer (Lê Hồng Phương)
  https://github.com/phuonglh/vn.vitk
- Dongdu (Lưu Anh Tuấn): SVM
  http://viet.jnlp.org/dongdu
- Pyvi (Trần Việt Trung): https://github.com/trungtv/pyvi
- Từ điển từ:
  - http://tratu.coviet.vn/tu-dien-lac-viet.aspx
  - http://tratu.soha.vn/
  - https://www.informatik.uni-leipzig.de/~duc/Dict/

Một số công cụ tách từ

- Bài tập: cài đặt và chạy chương trình tách từ Pyvi

```
1  from pyvi import ViTokenizer
2
3  ex1 = "thời khóa biểu đang được cập nhật"
4  ex2 = "môn học xử lý ngôn ngữ tự nhiên"
5  ex3 = "con ngựa đá con ngựa đá"
6  ex4 = "học sinh học sinh học"
7  ex5 = "Tách từ là bài toán nhận diện từ trong văn bản tiếng Việt"
8
9  if __name__ == "__main__":
10     print (ViTokenizer.tokenize(ex5))
```